

Novosibirsk State University

Backgammon
Documentation
Digital Platforms course project

Group 25214
Dedov Dmitry
Starikova Maria
Medvedev Mikhail

Contents

1. Introduction.....	3
1.1. Project overview.....	3
1.2. Backgammon rules.....	3
2. Hardware design.....	5
2.1 LED Matrix system.....	5
2.1.1 Video controller.....	6
2.1.2 Memory wrapper.....	7
2.2 Dice.....	8
2.2.1 Random controller.....	9
2.3 I/O.....	9
2.4 Game state controller (turn changer).....	11
3. Software.....	12
3.1 Entry point.....	12
3.2 Pseudo random generator.....	13
3.3 Terminal control functions.....	14
3.4 Board modification functions.....	15
3.5 Player move.....	16
3.6 Computer move.....	19
3.6.1 Move evaluating.....	22
3.7 Logic.....	23
3.8 Game initialization.....	27
4. Project Structure.....	28
5. Quick start.....	29

1. Introduction

Our project is a full implementation of the classic board game “Long Backgammon” with one simplification – each player has 11 checkers instead of the standard 15.

1.1. Project overview

We executed the following key features:

- Full set of rules: standard movement of checkers on two dice, doubles, no hitting, head and block rules, bearing off
- Game process: alternate moves with indication of the current player
- End of the game: when one of the players bears off all 11 checkers
- LED matrix visualization: game state displayed on a 25x32 dot matrix, different colors for white (player) and black (computer) checkers
- Keyboard input: a-x keys to select a point, z key to pass a move, y key to bear off
- Text terminal interface: output of game status, dice values, errors and debug information
- Random number generator: unpredictable dice values

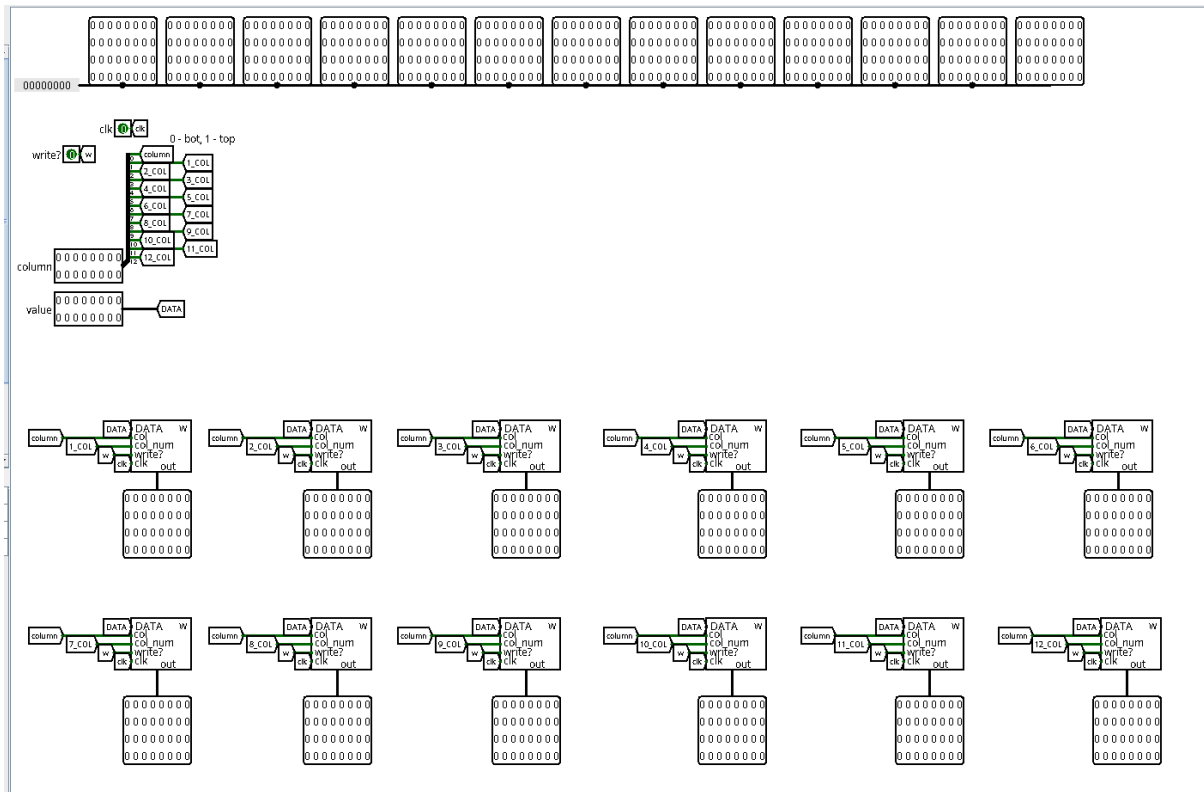
1.2. Backgammon rules

The purpose of the game is to move all your checkers counterclockwise to “home” and bear them off the board faster than the opponent.

1. Starting points: Each player’s 11 checkers are placed on their starting point (“head”)
2. Start of the game: The human player makes the first move

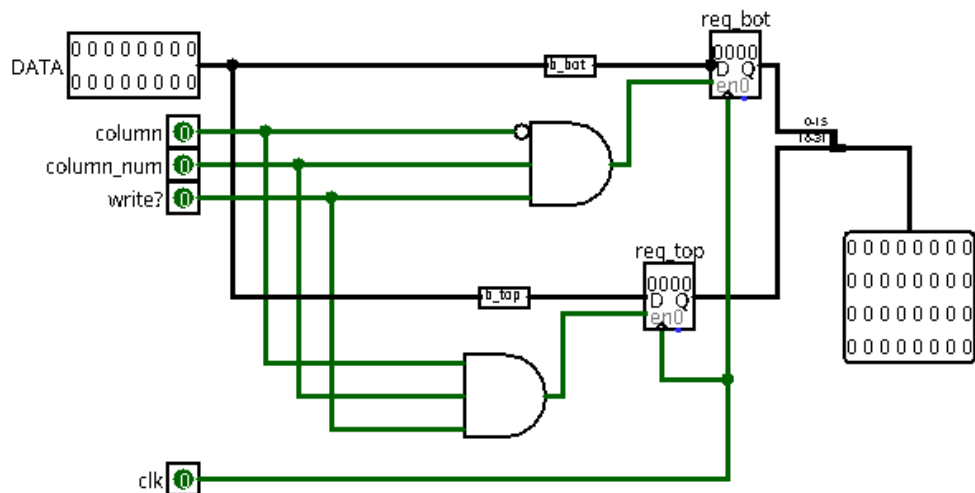
3. Movement: In one move the player can take one or two checkers using the values of two dice. He can place them on open points or on points with his checkers.
4. Doubles: If the same number appears on the dice, the player makes 4 moves on that number
5. Pass: If the player can't make a move, he passes (manually)
6. Head rule: Only one checker can be taken from the starting point in one move (two if the player rolls doubles)
7. Prime rule: The player can build a "prime" of 6 checkers in a row to block the opponent's checkers. It is prohibited to block all the opponent's checkers or to place 6 checkers if there is no opponent's checker in the front unless he has already started bearing off. (in our project you can find it by name "zabor rule")
8. No hitting: It is forbidden to place checkers on points occupied by the opponent's checkers
9. Bearing off checkers: It begins when all 11 checkers have been moved to the last quarter of the board ("home"). Checkers can be borne off if the die shows the number of steps required to exit. If the point is free, the player can use a bigger die value to bear off a checker from a distant point unless there are checkers between.
10. End of the game: The first player to bear all his checkers off the board wins

2.1.1 Video controller



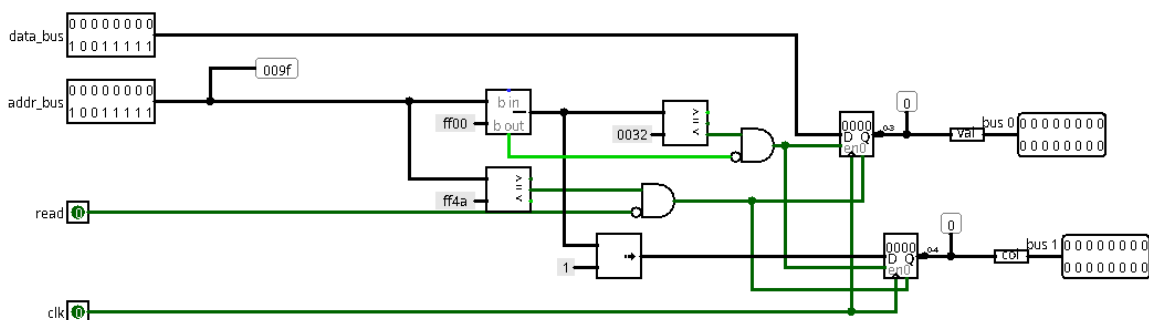
The video controller has 4 inputs: clock, write, columnn, value. A clock generator is connected to the clock, the write flag is responsible for the operation of the controller at the moment, if it is raised, the controller receives and processes data from the columnn and value inputs, if not, it ignores. The columnn and value inputs accept a *16-bit array*. The columnn array can have a maximum of 2 bits equal to one, the zero bit is responsible for selecting the part of the field: 0 - bottom, 1 - top, then the next bit is responsible for selecting the columnn, it is determined by the position of the bit in the array. takes an array of bits, where the number of bits equal to one is equal to the number of checkers that will be drawn in the columnn.

Each of the 12 columns is controlled by the following element:



It consists of 2 registers, each of which is responsible for its own half of the playing field. `b_top` and `b_bot` (`w_top` and `w_bot` for white checkers) controllers shift the value for correct rendering on the matrix.

2.1.2 Memory wrapper

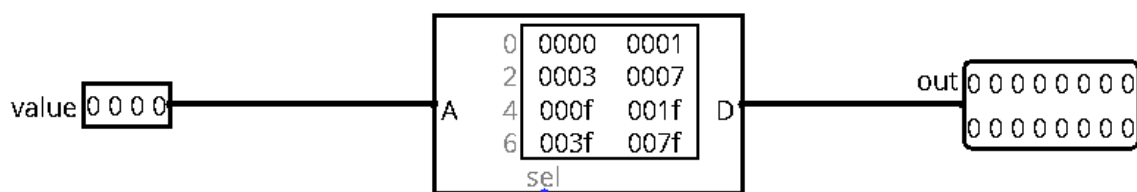


Before the information reaches the video controller, it passes through the memory wrapper.

This chip also has 4 inputs: `data_bus`, `address_bus`, `clock`, `read`. This chip constantly listens to the data and address bus and detects writes to the memory area that is responsible for the number of chips in the matrix

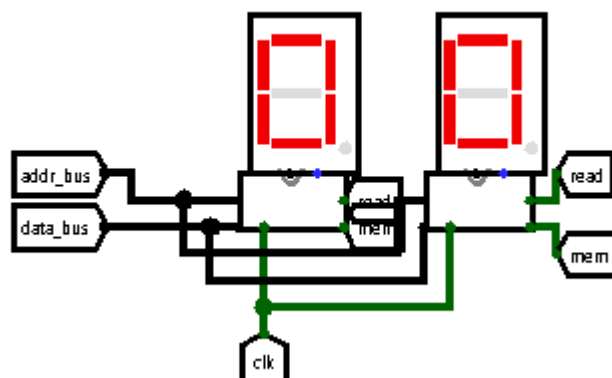
(0xff00 - 0xff31). When the addresses match, the chip reads the data_bus and extracts from there how many checkers should be written to the matrix in binary form, then converts them into a bit array with which the video controller can work, also subtracting the address of the beginning of the memory area allocated for interaction with the matrix, determines the column and also converts this information into a bit array, then all this information goes directly to the video controller.

The value and column converters are just plain ROM.



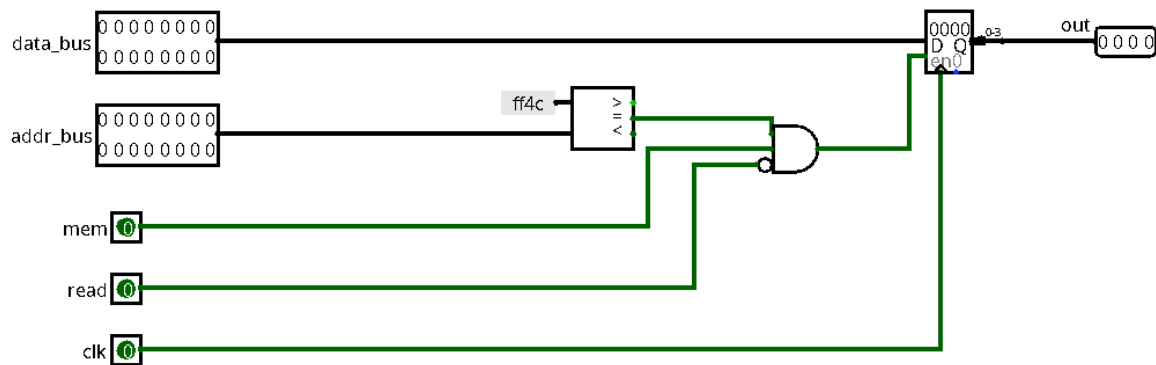
The chip also detects turn changes to clear the registers.

2.2 Dice



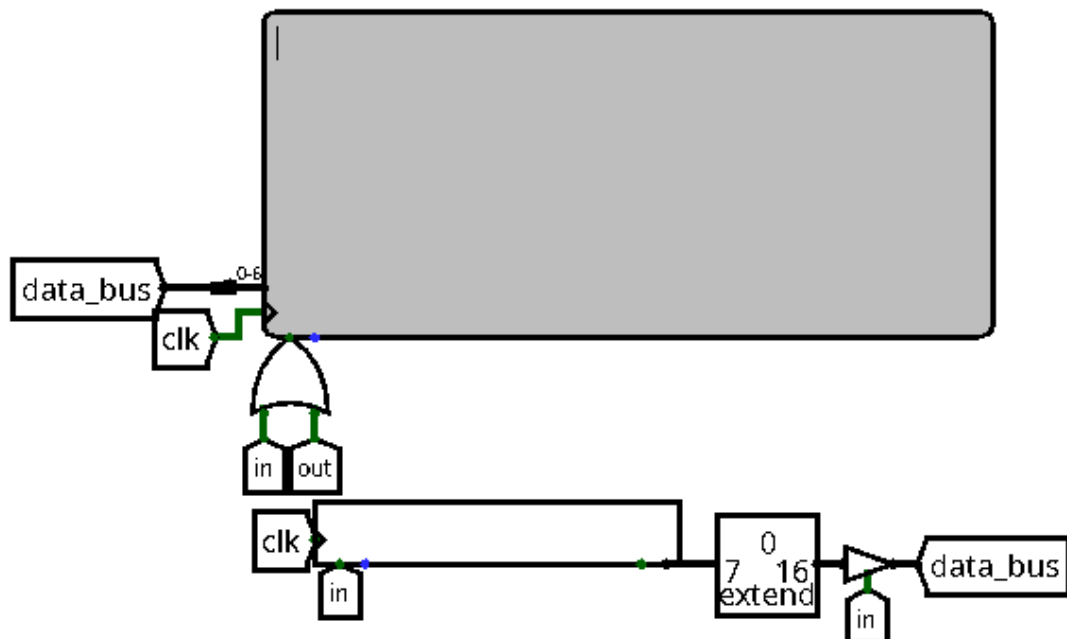
The dice are rendered using two hex digit displays. Their work is also provided by controllers.

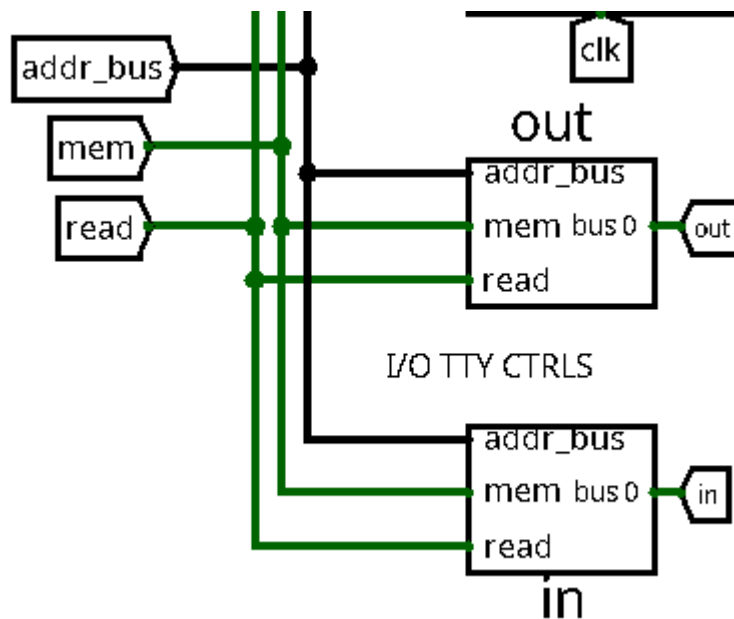
2.2.1 Dice display controller



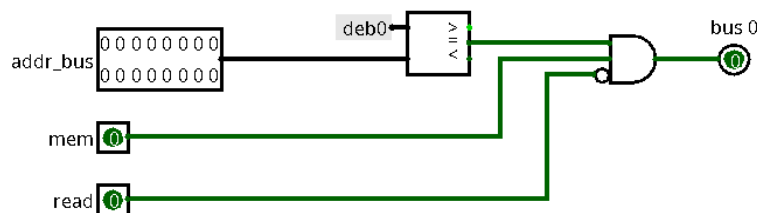
The principle of operation is very similar to memory wrapper. It also listens to address and data buses and, when writing to the memory area allocated for the dice (`0xff4c`, `0xff4d`), stores the data and displays it on the display.

2.3 I/O

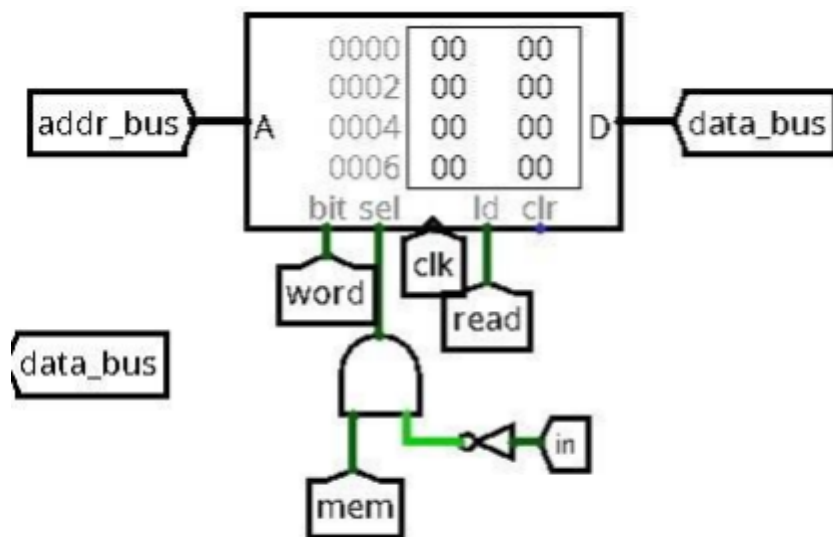




Player input is organised using the keyboard and TTY terminal. When a symbol is pressed on the physical keyboard of the computer, it is detected by the Logisim keyboard module. When it's time for the processor to read user input, it sets the **address_bus** to **0xdeb0** which activates the **in** signal and input character is automatically written to the data bus where the terminal reads them and prints one by one on clock signals. It's also possible for the processor to write text to the terminal which it does on address **0xdeae**. To avoid signal conflicts with the memory chip which also listens to the address bus, memory gets blocked by sending 0 to its chip select input signal. It is achieved using the AND gate and inverted **in**.

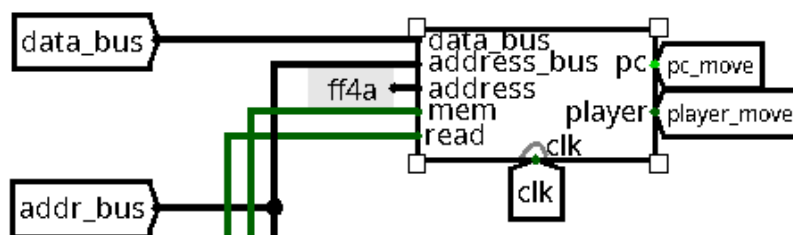


OUTPUT controller

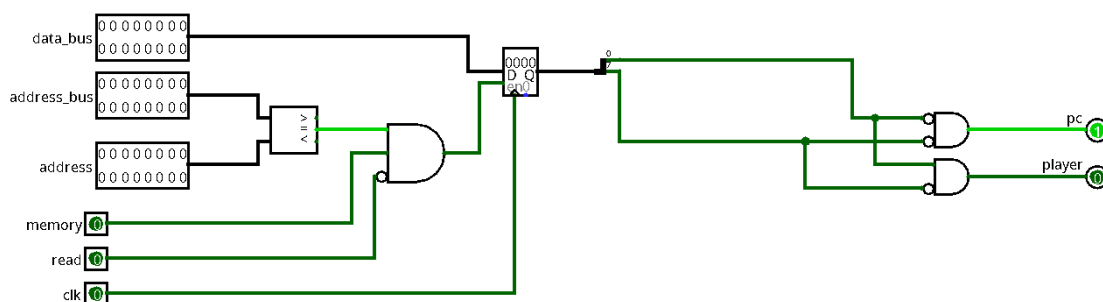


memory chip in the circuit.

2.4 Game state controller (turn changer)



GAME STATE CTRL



It also works on a principle similar to the memory wrapper. Listens to the address and data bus, detects a turn change (when the `_player` variable changes at `0xff4a`) and, depending on the received data, raises `pc_move` or `player_move`.

3. Software

The software part is written in C language.

3.1 Entry point

```
...
43  main: ext
42  asect 0
41  run: ext
40  default_handler: ext    # as external
39
38  # Interrupt vector table (IVT)
37  # Place a vector to program start and map
36  # all internal exceptions to default_handler
35  dc run, 0                # Startup/Reset vector
34  dc default_handler, 0    # Unaligned SP
33  dc default_handler, 0    # Unaligned PC
32  dc default_handler, 0    # Invalid instruction
31  dc default_handler, 0    # Double fault
30  align 0x80               # Reserve space for the rest
29  | | | | | |             # of IVT
28
27  # Exception handlers section
26  rsect exc_handlers
25
24  # # This handler halts processor
23  default_handler>
22  |   halt
21  |
20  # # Main program section
19  rsect run
18
17  run>                                # -- Begin function main
16  |   addsp -300
15  |   jsr main
14  |
13  asect 0xff00
12  _points> ds 50 # 0xff00
11  _colors> ds 24 # 0xff32
10  _player> ds 1 # 0xff4a
9   _reserved> ds 1 # 0xff4b
8   _random> ds 2 #0xff4c,0xff4d
7   _amt_of_checkers> ds 2 #0xff4e, 0xff4f
6
5  end.
```

The entry point to the program is the run.asm file

This file initializes the entire program, specifies the variables that need to be accessed from the schema.

3.2 Pseudo random generator

```
unsigned int state = 44257;

unsigned int xorshift16() {
    unsigned int x = state;
    x ^= x << 7;
    x ^= x >> 9;
    x ^= x << 8;
    state = x;
    return x;
}

void randomize(){
    for(int i = 0; i < 2; i++){
        unsigned int val;
        do {
            val = xorshift16() & 0x07;
        } while (val < 1 || val > 6);
        _random[i] = (char)val;
    }
}
```

In order to simulate dice during the game the pseudo-random number generator is needed. At the heart of it is the **state** global variable initialized with some number. To get the next pseudo-random number the **xorshift16()**

function is applied to the variable updating its value. It XORs it with itself shifted 7 bits to the left. Then again and again with itself shifted 9 bits to the right and 8 bits to the left correspondingly. There is also the **randomize()** function which fills the **_random[2]** array with normalized random values obtained using **xorshift16()** (the normalization is needed because physical dice can only yield values 1 - 6. If the obtained value is invalid, it's re-obtained). It simulates one dice roll.

volatile char* _random[2] array is the memory space from where digit displays take values of the dice to be output.

3.3 Terminal control functions

```
6 void print_to_tty(char* string){
7     while (*string){
8         *out = *string;
9         string++;
10    }
11 }
12
13 unsigned char getc(){
14     unsigned char chr;
15     while(1) {
16         state ^= (state << 1);
17         chr = *in;
18         if (chr >= 'a' && chr <= 'z') {
19             return chr - 'a';
20         }
21     }
22 }
23
```

There are two functions used to obtain user input and output game information (except dice values). Both use **in** and **out** pointers containing addresses of the corresponding IO devices. They belong to the lowest layer of abstraction. Other functions used within the game use them to manipulate IO.

- 1) **void print_to_tty(char* string)**. Where string is null-terminated. Prints string to the TTY terminal character by character.
- 2) **unsigned char getc()** - waits for a character in the infinite loop. Returns the 0 to 25 index of the letter in the English alphabet. It only breaks the loop if the input character is a lowercase letter. It also changes the seed so that the game is non-deterministic.

```
2 volatile char* out = (char*)TTY;
3 volatile char* in = (char*)KEYBOARD;
4
```

3.4 Board modification functions

To the same level of abstraction belong the following functions used to manage the board.

```
5  char move_checker(unsigned char* move){
6      char color = _colors[move[0]];
7      if (_points[move[1]+1]++ == 0){
8          _colors[move[1]] = color;
9      }
10     if(--_points[move[0]+1] == 0){
11         _colors[move[0]] = 0;
12     }
13     return 0;
14 }
15
16 char remove_checker(unsigned char from){
17     if(--_points[from+1] == 0){
18         _colors[from] = 0;
19     }
20     _amt_of_checkers[_player]--;
21     return 0;
22 }
23
```

- 1) **move_checker** - modifies numbers of checkers in move[0] and move[1] columns (from and to).
- 2) **remove_checker** - deletes the checker from position from and decrements the amount of checkers that are left.

_points[25] stores the number of checkers currently on each of the 24 columns (column numbers start from 1).

_amt_of_checkers[2] stores the number of checkers of the player and the computer at the moment

_player stores the side which is acting at the moment.

move[2] is a pointer to the arrays storing information about move in the following form - [from, to].

`_colors[25]` stores which side the column belongs to.

```
extern volatile char _player; // 1 - player move; 0 - computer move
```

```
extern volatile unsigned int _points[25]; // zero is null point, starts from 1
extern volatile char _colors[25]; // 0 - clear point, 1 - player's point, 2 - computer's point
```

`_points`, `_colors` arrays and `_player` variable belong to the memory space shared among the processor and the board device. board updates itself based on them. Thus they are declared with **volatile** keyword.

3.5 Player move

```
void player_move(unsigned char* can_remove_checker){
    unsigned char move[2];
    print_to_tty("\n--- Player Turn ---\n");
    randomize();
    _player = 1;

    if (!(*can_remove_checker)){
        if (is_all_in_home()){
            print_to_tty ("\nDBG: all checkers in home\nnow you bear off them");
            *can_remove_checker = 1;
        }
    }

    // _random - physical array, which you can see on displays
    // dice - actual array, which help us with doubles

    unsigned char dice[4];
    int dice_count = 0;
    int head_can_taken = 0;

    char d1 = _random[0];
    char d2 = _random[1];

    if (d1 == d2) {
        print_to_tty("\nRolling doubles!!");
        dice[0] = d1; dice[1] = d1; dice[2] = d1; dice[3] = d1;
        dice_count = 4;
        head_can_taken = 2;
    } else {
        dice[0] = d1; dice[1] = d2;
        dice_count = 2;
        head_can_taken = 1;
    }
}
```

The `player_move` function is responsible for the player's move, which takes one argument – a pointer to a byte (`can_remove_checker`) that indicates

whether we have all the checkers in our house and whether we can make a bear off.

First, at the start of a turn, the dice are rolled, as described in the **pseudo-random** section. Then, if the **can_remove_checker** flag is down, we check whether all checkers have entered the house with **all_in_home** function. This function will be described in the logic section; if so, we up the flag and make a signal that it is possible to start a bear off. Next, we process the dice that we receive from **randomize**; if the values that we receive are equal, then we record the double as **dice_count = 4**, else **dice_count = 2**, after that we transfer all our values to the dice array. Also, the double allows us to take two checkers from head in one move; this information is stored in the **head_can_taken** variable. The **dice** array is designed to prevent us from having artifacts on displays while processing a move.

```

You, 3 weeks ago • player move general changes
while (dice_count){
    print_to_tty("\nMoves left: ");
    *(char*)TTY = dice_count + '0';

    // * fetch coordinates
    print_to_tty("\nfrom (z to pass): ");
    move[0] = getc();
    if (move[0] == 'z' - 'a') {
        print_to_tty("\nPassed.\n");
        break;
    }

    if (*can_remove_checker) print_to_tty("\nto: (y to bear off) ");
    else print_to_tty("\nto: ");
    move[1] = getc();

    if (move[1] == 24){
        if (!(*can_remove_checker)){
            print_to_tty("You cannot bear off now");
            continue;
        } else{
            int used_dice = validate_bear_off(move[0], dice, dice_count);
            if(!used_dice) continue;
            remove_checker(move[0]);
            for(int i = 0; i < dice_count; i++){
                if (dice[i] == used_dice) {
                    dice[i] = dice[--dice_count];
                    break;
                }
            }
            if (d1 != d2) {
                if (_random[0] == used_dice) _random[0] = 0;
                else if (_random[1] == used_dice) _random[1] = 0;
            } else if (dice_count == 2) _random[1] = 0;
        }
    }
} else {

```

Next, the player moves as long as he has available die, or skips his turn.

First, let's consider the case when all the player's checkers have reached home and he can make bear off. This happens when the player points cell y in the direction of the turn. Using the **validate_bear_off** function (it will also be discussed in the logic section), we check whether we can do this and what die, if any, we save it to the **used_dice** variable. Next, using **remove_checker**, the checker is removed and the used die are searched and deleted from the **dice** array (when a used die is found, the most recent one from the list is copied to its place, and the total number of dice decreases).

```

1  } else {
2      // normal move, when can_remove_checker down
3      print_to_tty("\nMove validation...");
4
5      if (is_move_valid(move, dice, dice_count, head_can_taken)){
6          move_checker(move);
7          if (!zabor_rule()){
8              unsigned char undomove[2] = {move[1], move[0]};
9              move_checker(undomove);
10         } else {
11             print_to_tty("\nok...");
12             if (move[0] == 0) head_can_taken--;
13
14             int dist = get_dst(move[0], move[1], _player);
15             for(int i = 0; i < dice_count; i++){
16                 if (dice[i] == dist) {
17                     dice[i] = dice[dice_count - 1];
18                     dice_count--;
19                     break;
20                 }
21             }
22
23             if (d1 != d2) {
24                 if (_random[0] == dist) _random[0] = 0;
25                 else if (_random[1] == dist) _random[1] = 0;
26             } else if (dice_count == 2) _random[1] = 0;
27         }
28     } else {
29         print_to_tty("\nInvalid\n");
30     }
31 }
32
33 _player = -1;
34 }

```

In a normal situation, first using `is_move_valid` (logic section) it is checked whether the move is valid and whether we can move like this. Next, a move is made and it is checked whether the `zabor` rule is violated, if so, the move is canceled and the player is invited to move again. Also, if the move is made from the head, the `head_can_taken` is updated. Die removal occurs similarly to the first case.

At the end of the turn, the `_player` flag is reset to -1

3.6 Computer move

The basic points of move are similar to the `player_move` function, so only differences are described.

```

while (dice_count){
    int bst_score = -1;
    char bst_from = -1;
    char bst_to = -1;

    char best_is_bear_off = 0;
    int dice_for_bear_off = 0;

    for (int i = 0; i < dice_count; i++) {
        int curr = dice[i];

        for (char from = 0; from < 24; from++) {
            if (_colors[from] != 2) continue;

            char to = from + curr;
            char is_bear_off = 0;

            if (from >= 12) {
                if (to >= 24) to -= 24;
            } else {
                if (to >= 12) {
                    if (*can_remove_checker){
                        to = 24;
                        is_bear_off = 1;
                    } else continue;
                }
            }

            move[0] = from;
            move[1] = to;
        }
    }
}

```

The computer's logic is based on a greedy algorithm (We tried to make the check 2 moves ahead, but even it is very slow in some situations.). For each available checker, the computer calculates the most advantageous valid move using the **evaluate_move** (It will be discussed further.) function.

```

        if (is_bear_off) {
            int used_dice = validate_bear_off(from, dice, dice_count);
            if (used_dice) {
                int score = evaluate_move(from, to, 1);
                if (score > bst_score) {
                    bst_score = score; bst_from = from; bst_to = to;
                    best_is_bear_off = 1;
                    dice_for_bear_off = used_dice;
                }
            }
        } else {
            if (is_move_valid(move, dice, dice_count, head_can_taken)) {
                int score = evaluate_move(from, to, 0);
                move_checker(move);
                if (zabor_rule()) {
                    if (score > bst_score){
                        bst_score = score; bst_from = from; bst_to = to;
                        best_is_bear_off = 0;
                    }
                }
                unsigned char undomove[2] = {to, from};
                move_checker(undomove);
            }
        }
    }

    if (bst_score == -1) {
        print_to_tty("\nComputer passed.");
        break;
    }
}

```

Further logic copies the logic of `player_move`.

3.6.1 Move evaluating

```
static int evaluate_move(char from, char to, char is_bear_off) {
    int score = 0;

    if (is_bear_off) return 1000;

    if (from == 12) score += 80;

    if (_points[to + 1] == 0) {
        score += 60;

        if (to >= 0 && to <= 5) score += 40;
    }
    else {
        if (_points[to + 1] >= 2) score -= 60;
        else score += 10;
    }

    if (to > 0 && _colors[to - 1] == 2) score += 50;
    if (to < 23 && _colors[to + 1] == 2) score += 50;

    return score;
}
```

The main function of the bot is **evaluate_move**. With its help, you can fully customize the behavior of the bot, from peaceful to an aggressive configuration. The current settings force the bot to move forward and try to occupy free cells.

3.7 Logic

```
3 char is_all_in_home() {
4     if (_player == 1) {
5         _player = -1;
6         for (int i = 1; i < 19; i++) {
7             if (_colors[i-1] == 1 && _points[i] > 0){
8                 _player = 1;
9                 return 0;
10            }
11        }
12        _player = 1;
13    } else {
14        _player = -1;
15        for (int i = 12; i < 24; i++) {
16            if (_colors[i] == 2 && _points[i + 1] > 0){
17                _player = 0;
18                return 0;
19            }
20        }
21        for (int i = 0; i < 6; i++) {
22            if (_colors[i] == 2 && _points[i + 1] > 0){
23                _player = 0;
24                return 0;
25            }
26        }
27        _player = 0;
28    }
29    return 1; // all in home
30 }
```

Let's start with the `all_in_home` function, which allows us to find out if all the checkers have arrived in the house or not. It is called every time both sides move until the `can_remove_checker` flag rises. Before starting work, it checks who is making a move at the moment and resets the `_player` flag to -1 to prevent artifacts from appearing in the matrix. It runs through all fields that do not belong to the home and if it finds a checker, it restores `_player` and returns 0, if no checkers were found, it returns 1

```
int get_dst(char from, char to, int player) {
    int dst = to - from;
    if (player == 1) {
        if (dst <= 0) return -1;
        return dst;
    } else {
        if (from < 12 && to >= 12) return -1;
        if (dst < 0) dst += 24;
        if (dst == 0) return -1;
        return dst;
    }
}
```



```

1 char is_move_valid(unsigned char* move, unsigned char* dice, int dice_count, int head_can_taken){
2     int color = (_player == 1) ? 1 : 2;
3     if (_colors[move[0]] != color || !_points[move[0]+1]){
4         if (color == 1) print_to_tty("\nErr: Not your checker!\n");
5         return 0;
6     }
7
8
9     char opponent = (_player == 1) ? 2 : 1;
10    if (_colors[move[1]] == opponent) {
11        if (color == 1) print_to_tty("\nErr: Point captured by opponent!\n");
12        return 0;
13    }
14
15    int dist = get_dst(move[0], move[1], _player);
16    if (dist <= 0 || dist > 6) {
17        if (color == 1) print_to_tty("\nErr: Invalid dist\n");
18        return 0;
19    }
20
21    char head_pos = (_player == 1) ? 0 : 12;
22    if (move[0] == head_pos && head_can_taken == 0) {
23        if (color == 1) print_to_tty("\nErr: Head rule\n");
24        return 0;
25    }
26
27    char can_move = 0;
28    for(int i = 0; i < dice_count; i++){
29        if (dice[i] == dist) {
30            can_move = 1;
31            break;
32        }
33    }
34    if (!can_move) {
35        if (color == 1) print_to_tty("\nErr: Dice doesn't exists\n");
36        return 0;
37    }
38
39    return 1;
40 }

```

The `computer_move` and `player_move` functions use `is_move_valid` to verify the validity of the move. This function checks whether you have correctly selected where you are moving from and where you are moving to, whether the field is occupied by the opponent, whether the distance of the move (1-6) is correct and whether there is a die to move like this, using the `get_dst` function, and it is possible to take from the head. If all checks pass, it returns 1, else 0.

```

31 char zabor_rule() {
30     if (_amt_of_checkers[!_player] < 11) return 1;
29
28     char opponent = (_player == 1) ? 2 : 1;
27     char color = (_player == 1) ? 1 : 2;
26     char opp_finish = (opponent == 2) ? 11 : 23;
25     for (int i = 0; i < 24; i++) {
24         int count = 0;
23         for (int j = 0; j < 6; j++) {
22             int idx = i + j;
21             if (idx >= 24) idx -= 24;
20
19             if (_colors[idx] == color)
18                 count++;
17             else
16                 break;
15         }
14         if (count == 6) {
13             char dst = get_dst(i+6, opp_finish, 0);
12             char found = 0;
11             // trying to find opponent chip ahead
10             for (int j = 0; j <= dst; j++){
9                 int curr = i+6+j;
8                 if (curr >= 24) curr -= 24;
7                 if (curr >= 24) curr -= 24;
6                 if (_colors[curr] == opponent){
5                     found = 1;
4                     break;
3                 }
2             }
1
48         if (!found) {
1             print_to_tty("\nErr: Illegal block (6 in row)\n");
2             return 0;
3         }
4     }
5     i += count;
6 }
7
8     return 1;
9 }

```

Next, **zabor_rule** is the heaviest function in logic, it checks the move for violation of the zabor rule (or prime rule). It goes through all the fields in the loop and counts the checkers. If it finds 6 in a row, it starts a forward pass and checks for the opponent's checkers, if there are none, it returns 0, which cancels the move. If the enemy has already started making bear off, the rules allow us to build zabor.

```

8 char validate_bear_off(unsigned char from, unsigned char* dice, int dice_count) {
7     int color = (_player == 1) ? 1 : 2;
6     if (_colors[from] != color || _points[from + 1] == 0) {
5         print_to_tty("\nErr: Not your checker!\n");
4         return 0;
3     }
2
1     // get dst
56 int dst = (_player == 1) ? (24 - from) : (12 - from);
1
2     // find dice
3     for (int i = 0; i < dice_count; i++) {
4         if (dice[i] == dst) {
5             return dst; // return dice
6         }
7     }
8
9     // check checkers behind
10 char checkers_behind = 0;
11 int home_start = (_player == 1) ? 18 : 6;
12 int player = _player;
13 _player = -1;
14 for (int i = home_start; i < from; i++) {
15     if (_colors[i] == player && _points[i + 1] > 0) {
16         checkers_behind = 1;
17         break;
18     }
19 }
20 _player = player;
21 // senior dice rule
22 if (!checkers_behind) {
23     int best_dice = 7;
24     for (int i = 0; i < dice_count; i++) {
25         if (dice[i] > dst && dice[i] < best_dice) {
26             best_dice = dice[i];
27         }
28     }
29     if (best_dice != 7) {
30         return best_dice;
31     }
32 }
33 print_to_tty("\nErr: No valid dice to bear off\n");
34 return 0;
35 }

```

The last function is responsible for verifying the correctness of bear off – **validate bear off**. First, the correctness of the checker selection is checked, then there is a search for a die that we can use. If no die are found, the senior rule is checked: you can use a larger dice only if there are no others behind the checker. If it is ok, next, the function will select the best die for the move.

3.8 Game initialization

```
int main(){  
  
    // init  
    _player = 1;  
    _amt_of_checkers[1] = 11;  
    _points[1] = 11;  
    _colors[0] = 1;  
    _player = 0;  
    _amt_of_checkers[0] = 11;  
    _points[13] = 11;  
    _colors[12] = 2;  
    _player = -1;  
    unsigned char rmv_chck = 0;  
    unsigned char ai_rmv_chck = 0;  
  
    // game loop  
    while(1){  
        player_move(&rmv_chck);  
        if (!_amt_of_checkers[1]) {  
            print_to_tty("\nPlayer win!");  
            break;  
        }  
        computer_move(&ai_rmv_chck);  
        if (!_amt_of_checkers[0]) {  
            print_to_tty("\nComputer win!");  
            break;  
        }  
    }  
}
```

When the program starts, the initial setup is performed, the chips are placed in positions, and the `can_remove_checker` flags for player and computer are initialized. Then an infinite loop starts until one of the players has played all of their chips (until the value in the array becomes zero)

4. Project Structure

```
|— app
|   |— ai.asm
|   |— graphics.asm
|   |— logic.asm
|   |— main.asm
|   |— test.asm
|— build
|   |— app.img
|— compiler
|   |— clang-19
|— logisim
|   |— plugins
|   |   |— logisim-banked-memory-0.2.2.jar
|   |   |— logisim-cdm-emulator-0.2.2.jar
|   |   |— logisim-debugger-0.2.2.jar
|   |— scheme.circ
|— src
|   |— ai.c
|   |— backgammon.h
|   |— graphics.c
|   |— logic.c
|   |— main.c
|   |— test.c
|— Makefile
|— README.md
|— run.asm
```

7 directories, 21 files

The source files of the project are located in the src directory.

The scheme and the plugins required for it are located in the Logisim directory.

The assembler code obtained using the compiler is located in the app directory.

The compiler is located in the compiler folder under the name clang-19.

The built image can be found in the build directory under the name app.img

5. Quick start

If you don't plan to build everything from source, go to step 2.

First, you need to download the C language compiler, this one was used in the project - <https://github.com/leadpogrommer/llvm-project-cdm>

After downloading, create a “compiler” directory and place the binary file there.

- 1) To build the project, recommended to use a makefile.

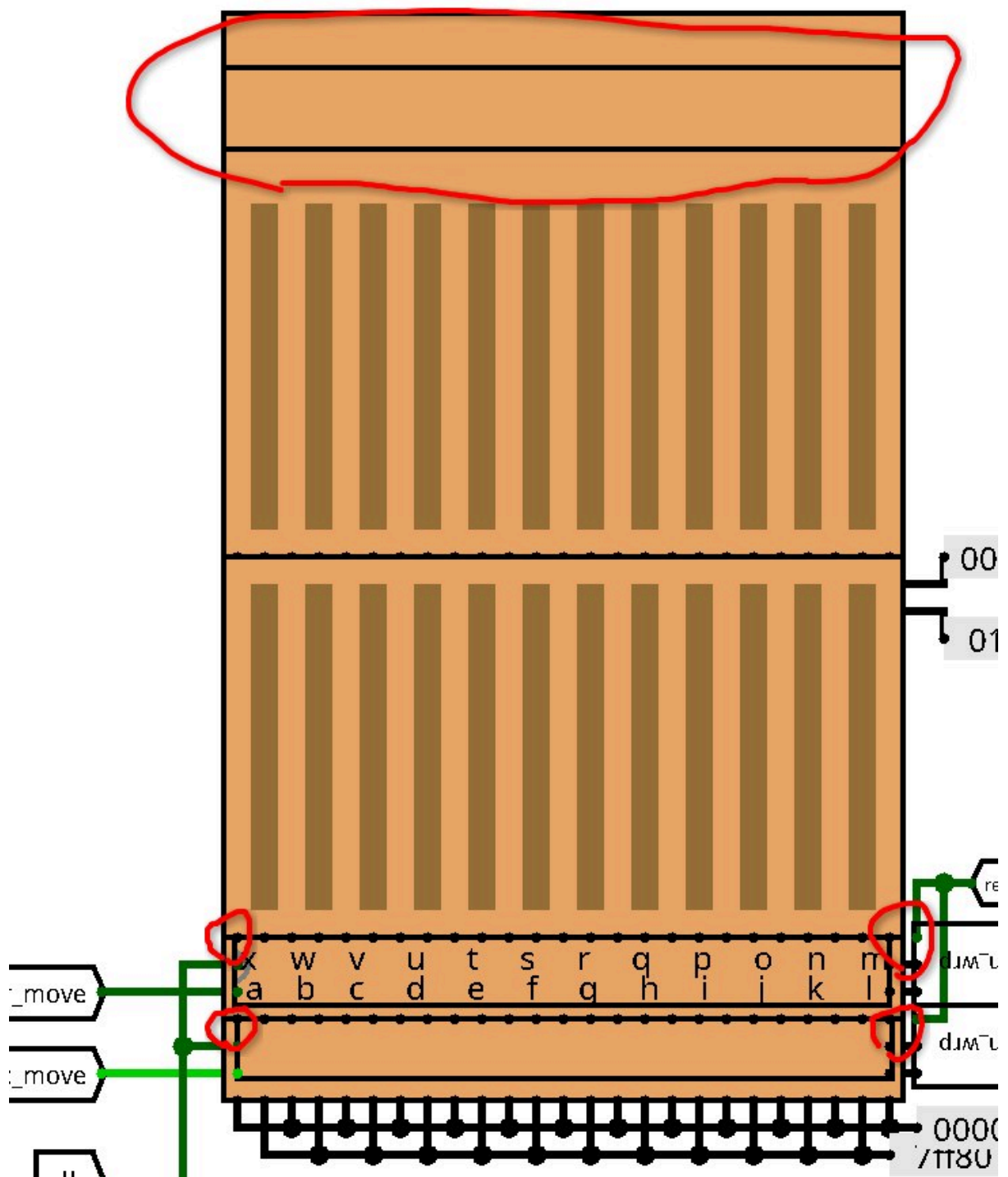
```
1  CLANG = ./compiler/clang-19
2  COCAS = ./env/bin/cocas
3  CFLAGS = -O2
4
5  GREEN = \033[1;32m
6  BLUE = \033[1;34m
7  RESET = \033[0m
8
9  SRCS := $(wildcard src/*.c)
10 ASM := $(SRCS:src/%.c=app/%.asm)
11
12 all: build
13     @printf "$(GREEN)✓ Success!$(RESET) Built with $(CFLAGS) to $(BLUE)build/app.img$(RESET)\n"
14
15 build: $(ASM)
16     @printf " - Building...\n"
17     @$(COCAS) -t cdm16 -o build/app.img \
18         run.asm \
19         app/ai.asm \
20         app/graphics.asm \
21         app/logic.asm \
22         app/main.asm
23
24 compile: $(ASM)
25     @printf "$(GREEN)✓ Success!$(RESET) Project compiled\n"
26
27 app/%.asm: src/%.c
28     @printf " - Compiling $(BLUE)$<...$(RESET)\n"
29     @$(CLANG) -c $< -S $(CFLAGS) -o $@
30
31 opt2: CFLAGS = -Wall -O2
32 opt2: all
33
34 opt3: CFLAGS = -Wall -O3
35 opt3: all
36
37 clean:
38     rm -f app/*.asm
39     @echo "Cleaned."
40
41 .PHONY: clean
```

1.1) to build run the make command in the project directory.

1.2) to convert C sources to CdM-16 assembly code execute

make compile command

- 2) To run a program in logisim you should open logisim, find the RAM module on main plate in it, right-click on it and then load image app.img from build directory to it.
- 3) Make sure the circuit opens correctly, the game field should look exactly like in the picture



! Recommended to use 8 version of java.